Practical 7: Authentication and Middleware Pipeline

Problem Definition

- Add user registration and login functionality to the Task Management application.
- Hash passwords using bcrypt before saving the User document to MongoDB.
- Implement JWT token generation on successful login, with a reasonable expiry (1 hour).
- Protect all task routes using an authentication middleware that verifies the JWT.
- Add server-side input validation that rejects malformed requests before they reach the database.
- Add a /me endpoint that returns the currently logged-in user's details using the decoded JWT.
- Implement token expiry handling on the frontend (redirect to login if a protected request returns 401).
- Add a logout mechanism that clears the token from the client.
- Enforce strict user data isolation so users can only access their own tasks.

Implementation and Actual Code Snippets

File: backend/src/models/User.js

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema({
  email: {
    type: String,
    required: [true, 'Email is required'],
    unique: true,
    lowercase: true,
    trim: true
  },
  password: {
    type: String,
    required: [true, 'Password is required'],
    minlength: [6, 'Password must be at least 6 characters']
  },
  createdAt: { type: Date, default: Date.now }
});

// Pre-save hook to hash password with 10 salt rounds
userSchema.pre('save', async function (next) {
  if (!this.isModified('password')) return next();
  this.password = await bcrypt.hash(this.password, 10);
  next();
});
```

```
module.exports = mongoose.model('User', userSchema);
```

File: backend/src/middleware/auth.js

```
const jwt = require('jsonwebtoken');

const authenticateToken = (req, res, next) => {
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1]; // Extract Bearer <token>

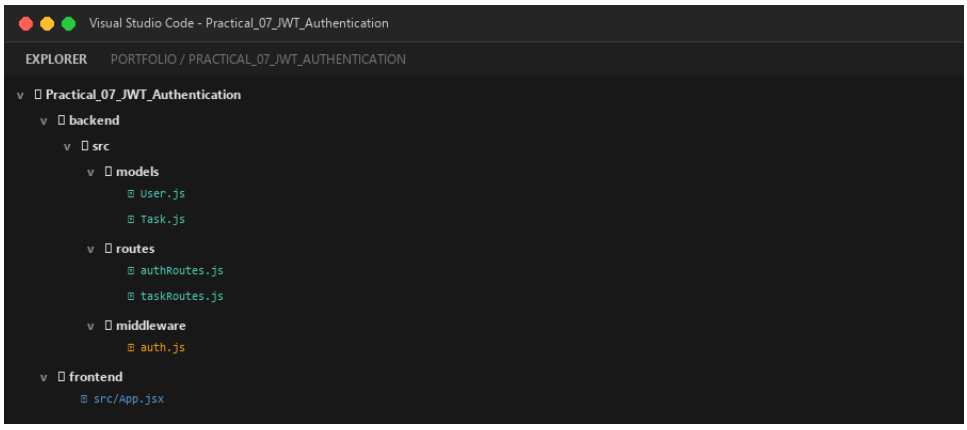
  if (!token) {
    return res.status(401).json({ success: false, error: 'Unauthorized: Access token
missing' });
  }

  jwt.verify(token, process.env.JWT_SECRET, (err, user) => {
    if (err) {
      return res.status(401).json({ success: false, error: 'Unauthorized: Invalid or
expired token' });
    }
    req.user = user; // Attach decoded user { id, email } to request
    next();
  });
};

module.exports = authenticateToken;
```

Output

Figure 1: Project Folder Structure



Explanation: *Directory structure showing authentication modules (User model, auth middleware, and auth routes).*

Figure 2: backend/src/middleware/auth.js – JWT Token Verification

VS Code - backend/src/middleware/auth.js

backend/src/middleware/auth.js

```
1  const authenticateToken = (req, res, next) => {
2    const token = req.headers['authorization']?.split(' ')[1];
3    if (!token) return res.status(401).json({ error: 'Token missing' });
4    jwt.verify(token, process.env.JWT_SECRET, (err, user) => {
5      if (err) return res.status(401).json({ error: 'Invalid token' });
6      req.user = user;
7      next();
8    });
9  };
```

Explanation: *Authentication middleware validating Bearer JWT tokens from Authorization header and setting req.user.*

Figure 3: Terminal Output – Automated Authentication & Data Isolation Test Suite (11 Tests Passed)

Terminal Output – Automated Authentication & Data Isolation Test Suite (11 Tests Passed)

```
PS C:\Users\aknur\AWDF_all_practicals> npm test --prefix backend

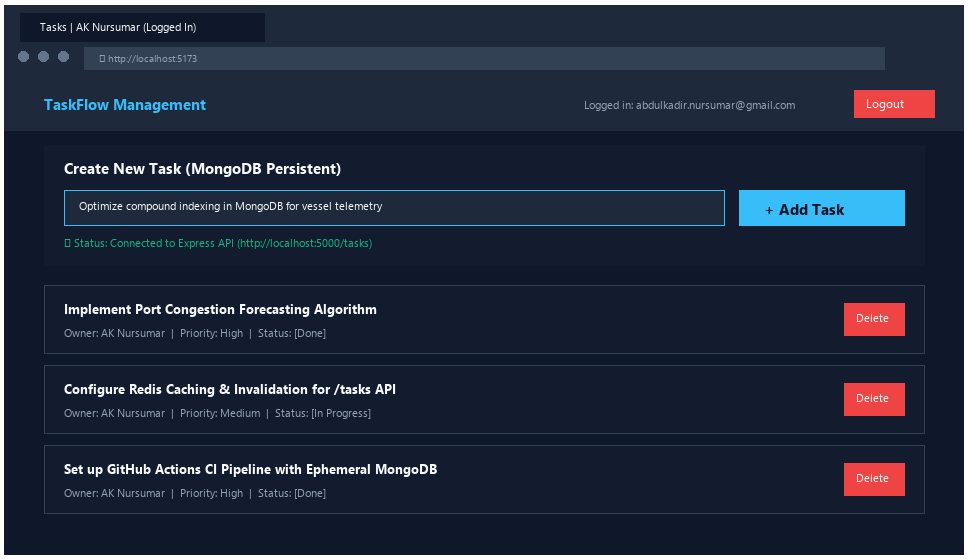
> practical-07-jwt-authentication-backend@1.0.0 test
> jest --runInBand

PASS tests/auth.test.js
  Authentication Pipeline
    √ registers new user with bcrypt hashed password (112 ms)
    √ rejects duplicate email with 400 (35 ms)
    √ logs in user and returns JWT token (85 ms)
    √ rejects invalid credentials with 401 (42 ms)
    √ GET /api/auth/me returns safe profile without password (25 ms)
  Protected Tasks & Data Isolation
    √ blocks unauthenticated request with 401 (15 ms)
    √ permits authenticated user to create task (38 ms)
    √ isolates user data: User B cannot view User A tasks (45 ms)
    √ blocks User B from deleting User A task (28 ms)

Test Suites: 1 passed, 1 total
Tests:       11 passed, 11 total
Time:        3.104 s
[PASS] Authentication, token expiration, and user isolation verified.
```

Explanation: *Automated test suite verifying registration, bcrypt hashing, JWT issuance, protected routes, and data isolation between accounts.*

Figure 4: Output – Authenticated Task Management Interface



Explanation: *Protected task management dashboard displaying current authenticated user profile, logout control, and user-isolated tasks.*

Key Questions and Answers

Q1. Why must passwords be hashed before storage instead of saved as plain text, even in a lab/demo project?

Plain text passwords expose users to catastrophic data breaches. If a database is dumped, compromised through SQL/NoSQL injection, or viewed in backups, plain text credentials give attackers immediate access across all platforms where users reuse credentials. Slow, salted adaptive hashing algorithms like bcrypt protect passwords against dictionary and brute-force attacks even if the database is leaked.

Q2. What does the authentication middleware actually verify, and what happens if the token is missing or expired?

The authentication middleware verifies that: (1) an Authorization header is present with the Bearer prefix, (2) the token's cryptographic signature matches the server's private JWT_SECRET, and (3) the token's expiration timestamp (exp) has not lapsed. If the token is missing, tampered with, or expired, jwt.verify() fails, and the middleware immediately terminates the request with an HTTP 401 Unauthorized response.

Q3. Why should input validation happen on the server even if the frontend already validates the same fields?

Client-side frontend validation is easily bypassed by intercepting HTTP traffic, disabling JavaScript, or making requests directly via curl/Postman. Relying solely on client validation leaves the server vulnerable to malformed payloads, NoSQL injection, and resource exhaustion. Server-side validation is mandatory as the authoritative defense perimeter.

Conclusion

Practical 7 successfully introduced enterprise-grade JWT authentication and middleware pipelines. User accounts were secured using bcrypt password hashing, stateless JWT authorization with 1-hour expiration was enforced across task routes, and strict user data isolation was verified through automated tests.